File Integrity Checker using SHA-256

Objective

To implement a file integrity checking system using the SHA-256 hashing algorithm in Python, which detects any modification in files by comparing hash values.

Theory

File integrity ensures that data remains unchanged and unaltered during storage or transmission. Cryptographic hash functions like SHA-256 generate a unique fixed-length hash for any input file.

- Even a small change in file content results in a completely different hash.
- SHA-256 produces a 256-bit (64 hexadecimal characters) hash.
- It is widely used in security applications due to its collision resistance.

Methodology

The implementation is divided into two parts:

1. Hash Generation (Generate_Hash.py)

- Reads files in binary mode.
- Processes data in chunks (4096 bytes).
- Computes SHA-256 hash for each file.
- Stores file names and corresponding hashes in a JSON file (`hashes.json`).

```
import hashlib
import json

def calculate_sha256(file_path):
    sha256 = hashlib.sha256()
    with open(file_path, 'rb') as f:
        while chunk := f.read(4096):
            sha256.update(chunk)
```

```python
    return sha256.hexdigest()

def generate_hashes(file_list, output_file="hashes.json"):
    hashes = {}

    for file in file_list:
        try:
            hashes[file] = calculate_sha256(file)
            print(f"[+] {file} hashed")
        except FileNotFoundError:
            print(f"[!] {file} not found")

    with open(output_file, 'w') as f:
        json.dump(hashes, f, indent=4)

    print(f"\nHashes saved to {output_file}")


# Example usage
files = ["file1.txt", "file2.txt"]
generate_hashes(files)
```

2. Hash Verification (Verify_hash.py)

- Reads stored hashes from `hashes.json`.
- Recomputes hash for each file.
- Compares current hash with stored hash.
- Displays result:
    - **OK (unchanged)** → File is intact
    - **MODIFIED** → File has been altered
    - **Missing** → File not found

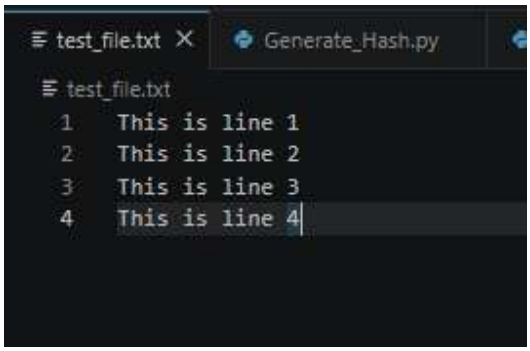
```python
import hashlib
import json

def calculate_sha256(file_path):
    sha256 = hashlib.sha256()
    with open(file_path, 'rb') as f:
        while chunk := f.read(4096):
            sha256.update(chunk)
    return sha256.hexdigest()

def verify_hashes(hash_file="hashes.json"):
    try:
        with open(hash_file, 'r') as f:
            stored_hashes = json.load(f)
    except FileNotFoundError:
        print("Hash file not found!")
        return

    for file, expected_hash in stored_hashes.items():
```

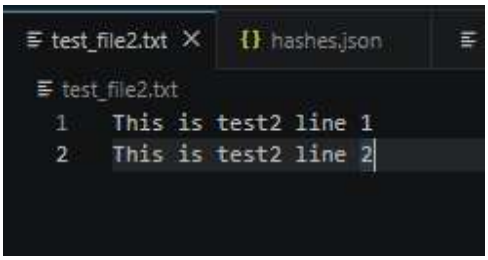
```
        try:
            current_hash = calculate_sha256(file)

            if current_hash == expected_hash:
                print(f" {file}: OK (unchanged)")
            else:
                print(f" {file}: MODIFIED")

        except FileNotFoundError:
            print(f" {file}: Missing")


# Example usage
verify_hashes()
```

Implementation Overview

Input Files

- `test_file.txt`
- `test_file2.txt`





Output File

- `hashes.json` containing SHA-256 hash values for each file

```
test_file2.txt   {} hashes.json ×   test_file.txt   Generate_Hash.py   Verify_hash.py
{} hashes.json > ...
1  {
2      "test_file.txt": "8d8ab71382a5d5365e413610849e794327a3bccf5ead8aaa2f578720648e3211",
3      "test_file2.txt": "beb3a803d67dfceb56b4ae13ac423a02b53e474858fbfb42731ce6daf25f73a5'
4  }
```

From the screenshots (Page 1 & 2):

- Files are created with sample text data.
- Hash values are generated and stored.

Verification script confirms integrity with output:
test_file.txt: OK (unchanged)
test_file2.txt: OK (unchanged)

```
[Running] python -u "c:\Users\HP\OneDrive\Desktop\File_Integrity\Verify_hash.py"
 test_file.txt: OK (unchanged)
 test_file2.txt: OK (unchanged)
```

Now let's alter the value of File1

```
test_file.txt ×   {} hashes.json   test_file2.txt
test_file.txt
1   This is line 1
2   This is line 2 altered
3   This is line 3
4   This is line 4
```

```
[Running] python -u "c:\Users\HP\OneDrive\Desktop\File_Integrity\Verify_hash.py"
 test_file.txt: MODIFIED
 test_file2.txt: OK (unchanged)
```

Results

- The system successfully generates and stores hash values.
- Verification accurately detects whether files are modified or unchanged.
- Demonstrates reliable file integrity checking using SHA-256.

Conclusion

The project effectively implements a file integrity checker using SHA-256 hashing. It ensures data authenticity and detects tampering efficiently. This approach is widely applicable in secure file storage, software distribution, and cybersecurity systems.